2/8/25

Task-4 : Use various datatypes, List, Tuples and Dictionary in python programming Key Terms covered: Datatypes, List, Tuple, Set, Dict

4.1 - List - Cafeteria sales

Aim:

Record a cafeteria, the sales for 7 days using a list; compute total and average sales find the best/worst day; and count how many days crossed a target.

Algorithm:

1. Start
2. Create an array list sales = []
3. for 7 days append integer sales to the list using append().
4. Compute total = sum(sales) and avg = total/7,
5. find max-val = max(sales), min-val = min(sales)
6. find corresponding days with index() (add +1 to convert my day number).
7. Count days above a target using count() on a boolean re map or with a loop
8. Stop

Program:

```
days=7
sales=[]
target=500
for s in range(8):
    sample_entries=int(input("Enter seven days sales count"))
    sales.append(sample_entries)
total=sum(sales)
avg=total/days
```

Output:

Enter seven days sales count 100

Enter seven days sales count 450

Enter seven days sales count 1200

Enter seven days sales count 589

Enter seven days sales count 98

Enter seven days sales count 348

Enter seven days sales count 900

Enter seven days sales count 239

Sales (Mon... Sun) : [100, 450, 1200, 589, 98, 348, 900, 239

Total : 3974

Average : 567.71

Best Day : 3 with 1250

Worst Day : 5 with 98

```
max_val = max(sales)
min_val = min(sales)
best_day = sales.index(max_val) + 1
worst_day = sales.index(min_val) + 1
print("Sales (Mon..Sun):", sales)
print("Total:", total)
print("Average:", round(avg, 2))
print("Best Day:", best_day, "with", max_val)
print("Worst Day:", worst_day, "with", min_val)
```

4.2 - Tuple - Lab timetable

Aim:

To manage and query an immutable daily lab slot schedule using a tuple, demonstrating membership checks, count(), index() and slicing

Algorithm:

1. Start
2. Define slots as a fixed tuple of integers
3. Read query hour
4. Check existence with a query in slots
5. Use count(), if positive, use index() to find the first position
6. Slice into morning and afternoon
7. Print results
8. Stop

Python Program:

```
slots = (9, 11, 14, 16, 14)
q_hour = 14
```

Output :

All lab slots : (9, 11, 14, 16, 14)

Is 14:00 present ? True

14:00 occurs 2 time(s)

First occurence position (1-based) : 3

Morning slots : (9, 11)

Afternoon slots : (14, 16, 14)

```
exists = (query in slots)
freq = slots.count(query)
first_pos = slots.index(query) + 1 if exists else "N/A"
morning = slots[:2]
afternoon = slots[2:]
print("All lab slots:", slots)
print(f"Is {query} : 00 present?", exists)
print(f"{query} : 00 occurs", freq, "time(s)")
print("first occurence position (1-based):", first_pos)
print("Morning slots:", morning)
print("Afternoon slots:", afternoon)
```

4.3 - Dictionary - Bookstore Billing

Aim:

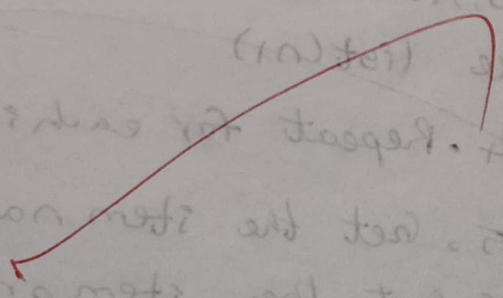
To manage a live price list and bill a customer using dictionary methods and views

Algorithm:

1. Start
2. Create an empty dictionary prices
3. Ask the user for the number of items in the price list(n)
4. Repeat for each item
5. Get the item name
6. Get the item price
7. Add the item to item and price to prices
8. Ask if

Output:

```
Enter number of items in price list: 3
Enter item name :box
Enter price of box : 15
Enter item name: pen
Enter price of pen: 10
Enter item name :pencil
Enter price of pencil : 5
Enter item to update prices (or Enter to skip): box
Enter new price for box : 20
Enter an item from price list (or Enter to skip) :pen
Available Items : ['box', 'pencil']
prices : [20.0 , 5.0]
costliest item : box at 20.0
Removed 'per' price (if existed ): 10.0
```



9. If the item exists in price. get the new price and update it

10. Find the costliest item by checking each item's price

11. Ask the user for an item to remove (or press Enter to skip)

12. If given, remove that item from prices

13. Show all available items, their prices, the costlies item and the removed item's price

14. Stop

Python program:

```
prices = {}
n1 = int (input ("Enter number of items in pricelist:
for _ in range(n1):
    item = input ("Enter item name:")
    price = float (input (f"Enter price of {item}:"
    prices[item] = price
rev_item = input ("Enter the item to update price (or
                   press Enter to skip):")
if rev_item in prices:
    new_prices = float (input (f"Enter new price for
                   {rev_item}:"))
    prices.update ({rev_item: new_price})
costliest_item = None
max_price = 0
for item, price in prices.items():
    if prices > max_price:
        max_price = price
        costliest_item = item
remove_item = input ("Enter an item to remove
                   from price list (or
                   press Enter to skip):")
removed_price = None
if remove_item:
    removed_price = prices.pop (remove_item
                                None)
```

```
print("In Available Items :", list(prices.keys()))
print("Prices :", list(prices.values()))
if costliest_item:
    print("Costliest item:", costliest_item, "at", max_pr
if remove_item:
    print(f"Removed '{remove_item}' price (if existed):
                                    removed_price)
```

4.4 - Set Techfest Participation

Two events, AI Hackathon and Robotics challenge have participants IDs stored in two sets. Add a late registered to AI Hackathon, remove a withdrawn participant from Robotics using discard(), then find participants in both events (intersection()), only in one (difference()), the total unique participants (union()).

Program:

```
ai_hackathon = set()
n1 = int(input("Enter number of participants in AI
                                    Hackathon:"))
for _ in range(n1):
    pid = input("Enter participant ID:")
    ai_hackathon.add(pid)
robotics_challenge = set()
n2 = int(input("Enter number of participants in
                                    Robotics challenge:"))
for _ in range(n2):
    pid = input("Enter participant ID:")
    robotics_challenge.add(pid)
late_id = input("Enter late registration ID for AI
                                    Hackathon (or press to skip)
```

Output:

Enter number of participants in AI Hackathon: 4

Enter participant ID: A1

Enter participant ID: A2

Enter participant ID: A4

Enter participant ID: A5

Enter number of participants in Robotics challenge:

Enter participant ID: A1

Enter participant ID: A2

Enter participant ID: A3

Enter participant ID: A7

Enter late registrant ID for AI Hackathon (or press Enter to skip): A8

Enter withdrawn participant ID from Robotics challenge (or press Enter to skip):

AI Hackathon: {'A1', 'A4', 'A5', 'A8', 'A2'}

Robotics challenge: {'A2', 'A7', 'A3'}

Both events: {'A2'}

Only AI: {'A4', 'A8', 'A5', 'A1'}

Only Robotics: {'A7', 'A3'}

Total Unique participants: 7

```
if late_id :
    ai_hackathon.add (late_id)
remove_id = input ("Enter withdrawn participant ID from
                    Robotics challenge (or press Enter to
if remove_id :                                  skip ): ")
    robotics_challenge.discard (remove_id)
both = ai_hackathon.intersection (robotics_challenge)
only_ai = ai_hackathon.difference (robotics_challenge)
only_robotics = robotics_challenge.difference (ai_hackathon
unique_all = ai_hackathon.union (robotics_challenge)
print ("\n AI Hackathon :", ai_hackathon)
print ("Robotics Challenge :", robotics_challenge)
print ("Both events :", both)
print ("Only AI :", only_ai)
print ("Only Robotics :", only_robotics)
print ("Total unique participants :", len (unique_all))
```

VEL TECH	
EX NO.	4
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	19

Result:

Thus the python program to use various datatypes, List, Tuples and Dictionary was executed and verified successfully.